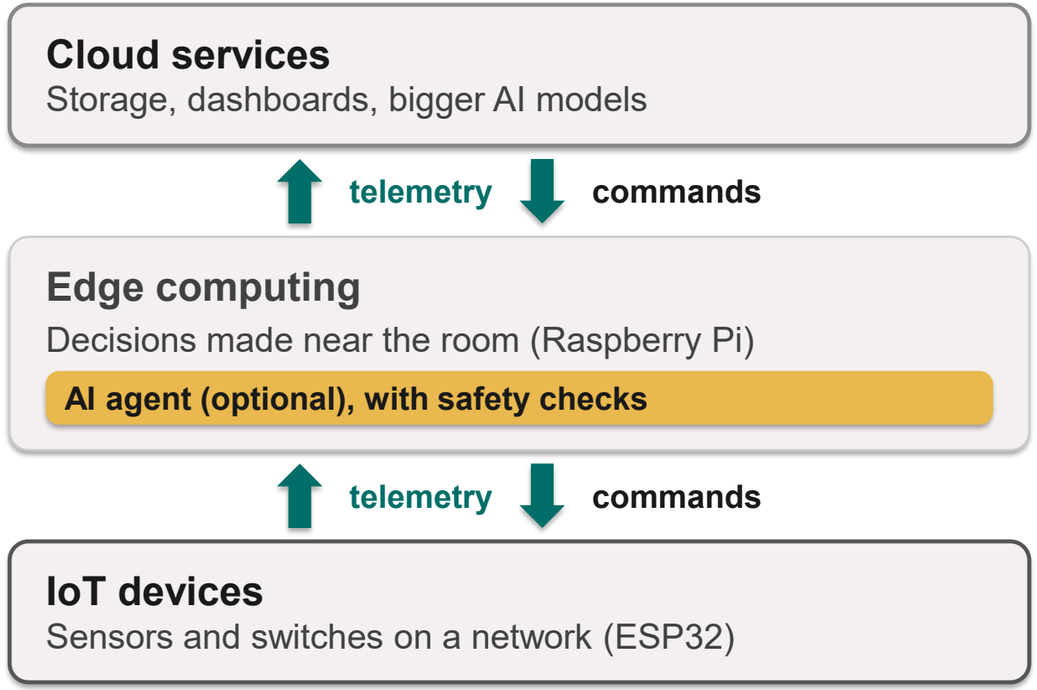
From IoT to Agentic AIoT

Sensors, small computers near the room, and smart automation

Welcome! Today we follow one classroom, Room 2.306, from a single sensor reading to a safe, smart AI agent.

By the end you can:

- Follow sensor data from the device to the cloud
- Explain MQTT (how devices send messages)
- Tell simple rules apart from AI agents
- Explain how to control machines safely



Lecture Map: Five Stages, One Room

SLIDES 3–10

1 The Basics

The room, sensors, readings, and the ESP32 chip.

How does the room become data?

SLIDES 11–18

2 Messaging

How devices talk (MQTT) and the Raspberry Pi nearby.

How does data travel, even when the internet fails?

SLIDES 19–22

3 Automation

Simple rules, where they fall short, and AI.

When are rules not enough?

SLIDES 23–29

4 Agents

AI agents, small vs big AI, the full system.

Where does an AI agent fit?

SLIDES 30–35

5 Practice

Staying safe, a full worked example, what to do next.

How do we keep it safe?

The story: the room makes data → the data travels → rules act on it → rules fall short, so an AI agent helps → we keep it safe.

Lecture Overview

- 1 The Basics
  The room, sensors, readings, and the ESP32 chip.
- 2 Messaging
  How devices talk (MQTT) and the Raspberry Pi nearby.
- 3 Automation
  Simple rules, where they fall short, and AI.
- 4 Agents
  AI agents, small vs big AI, the full system.
- 5 Practice
  Staying safe, a full worked example, what to do next.

Room 2.306: The Problem We Will Solve

The room: one classroom. It has a thermometer, a motion sensor that spots people, and a switch that turns cooling on or off (pretend, for the demo).

The goal: keep the room cool when people are inside. Turn cooling off when nobody is there, to save power.

The catch: it must still work when the internet goes down.

Comfort
People inside and hotter than 80 °F (26.7 °C): cool the room.

Efficiency
Nobody inside: cooling stays off.

Reliability
Keeps working with no internet.

Safety
Never send a dangerous command.

Observability
Write down every reading and action.

Built in stages: sense → message → rules → agent.

The Big Picture: Three Places

3 · Cloud (far away, on the internet)
Stores data for a long time, shows dashboards, runs big AI. Optional.

↑ readings ↓ settings and advice

2 · Raspberry Pi (in the building)
A small, cheap computer. It runs the mail room (MQTT broker), the rules, saved data, and an optional AI agent.

↑ readings ↓ commands

1 · ESP32 (in the room)
A small chip with Wi-Fi. It reads the sensors and flips the cooling switch.

Follow one reading

1 The sensor measures 27.4 °C.

2 The ESP32 sends it to the Pi.

3 The Pi sees it is hot and people are inside.

4 The Pi sends back: "cooling ON."

5 The ESP32 flips the switch.

The rest of the lecture zooms into each of these three places, one at a time.

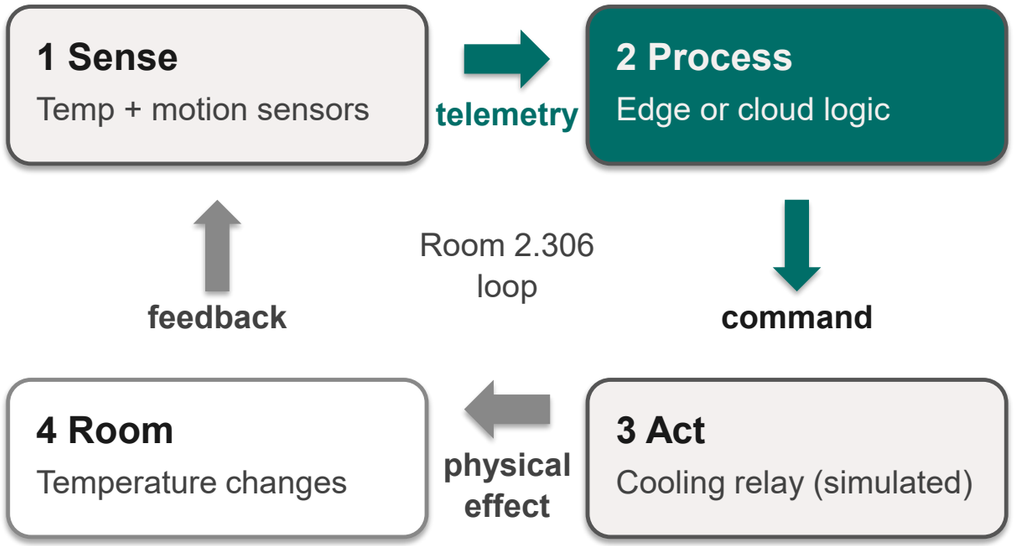
What Is the Internet of Things?

IoT (Internet of Things): physical devices that sense or act, networked so software can use their data.

Five parts: devices, connectivity, processing, applications, feedback.

Telemetry = data a device sends up by itself, like “it is 27 °C” (Greek: measuring from far away).

Commands go down: “turn cooling on.” An acknowledgment replies “done.”



Sensors, Actuators, and Embedded Devices

Temperature sensor
Heat → voltage or digital value

Motion sensor (PIR)
Passive infrared: movement → 0 / 1

telemetry signals

Microcontroller (embedded device)
A tiny computer on one chip. It runs one fixed program (firmware): reads sensors, then switches things on or off.

command signal

Actuator
The part that does the work: a switch, motor, or valve. Here it turns cooling on (pretend).

A reading is more than a number

Sampling: how often to measure

Units: state °C or °F explicitly

Timestamp: when it was measured

Calibration: correct offset and scale error

Quality: flag noise, gaps, out-of-range values

Two kinds of computer

	Microcontroller	General-purpose
Example	ESP32	Raspberry Pi
Software	Firmware / RTOS	Linux, apps
Typical job	Real-time I/O	Services, storage

RTOS = real-time operating system. I/O = input/output.

What One Sensor Reading Looks Like

A number alone is not enough. Each reading travels with extra facts, so software knows when it was taken and whether to trust it.

```
{
  "device":  "esp32-2.306",
  "ts":      "2026-10-07T14:32:05Z",
  "temp_c":  27.4,
  "occupied": true,
  "quality": "ok",
  "seq":     18342
}
```

Bad example

Just 27.4: °C or °F? When? Broken sensor? Which one? Nobody can tell.

ts = time

When the sensor took the reading, not when it arrived.

temp_c = temperature in °C

The unit is in the name, so nobody confuses °C and °F.

quality = can we trust it?

ok, stale (too old), or out_of_range (looks broken). Bad data gets ignored.

seq = message number

Counts up by 1. A skipped number means a lost message.

device = who sent it

Which sensor, in which room.

The ESP32: The Brain on the Device

Temperature sensor
Measures how hot it is.

Motion sensor
Detects if someone moves.

readings

ESP32

A small chip on the device. It has Wi-Fi built in.

1 Read the sensors

2 Send the data

3 Obey commands

on / off

Cooling switch
Turns cooling on or off (pretend). Its technical name is a relay.

Over Wi-Fi
The ESP32 sends readings to the MQTT broker, a mail room that passes messages along. It runs on a Raspberry Pi: a small, cheap computer in the building (slide 17). Commands come back the same way.

If Wi-Fi drops
Keep reading the sensors, keep the cooling switch in a safe setting, and try to reconnect.

What the ESP32 Does, Step by Step

```
every 5 seconds:
  read the sensors
  smooth the reading (average of last 5)
  mark it ok / stale / out of range
  send it

  if a command arrived:
    is it valid and within safe limits?
      yes: switch cooling on or off
    send back "done"

  if Wi-Fi is lost:
    keep a safe setting
    try to reconnect
```

How often to read
More often = fresher data but more power. Pick the slowest rate that is good enough.

Never get stuck
Waiting for Wi-Fi must not stop sensor readings or safety checks.

Watchdog
A built-in timer that restarts the chip if it freezes.

Safe setting
What the switch does when commands stop, for example cooling off after a timeout.

Lecture Overview

- 1 The Basics
  The room, sensors, readings, and the ESP32 chip.
- 2 Messaging
  How devices talk (MQTT) and the Raspberry Pi nearby.
- 3 Automation
  Simple rules, where they fall short, and AI.
- 4 Agents
  AI agents, small vs big AI, the full system.
- 5 Practice
  Staying safe, a full worked example, what to do next.

Two Different Jobs: Connecting and Messaging

Job 1 · Connecting

How the signal gets from A to B. Wi-Fi, Bluetooth, LoRaWAN. Like a phone line: it only carries the call.

Job 2 · Messaging

What is said on that line, and who hears it. MQTT: a device drops a note in a mailbox, and everyone who signed up for it gets a copy. HTTP: you ask one question and get one answer.

	Wi-Fi	Bluetooth Low Energy	LoRaWAN
Reach	A room or building	Same room	Up to several km
Power use	Higher	Low	Very low
Speed	Fast	Slow to medium	Very slow, tiny messages
Best for	Room 2.306 (what we use)	Wearables, nearby sensors	Far-away sensors that rarely report

Rough comparison: real range and power depend on hardware and surroundings.

MQTT: Publish–Subscribe Messaging

SENDERS		BROKER (MAIL ROOM) · TOPICS (MAILBOXES)		RECEIVERS
ESP32 sends readings	send	`room/2.306/telemetry` temperature, motion	deliver	**Pi services, agent** read the readings
Command sender Pi or agent; checked first	send	`room/2.306/command` orders, e.g. cooling on	deliver	**ESP32** reads the orders
ESP32 sends status and goodbye message	send	`room/2.306/status` online or not, and "done" replies	deliver	**Dashboard, Node-RED** read the status

Delivery level (QoS): 0 = send once, may be lost · 1 = retry until it arrives, may repeat · 2 = exactly once

Retained: broker keeps the last message so a newcomer sees it at once

Last Will: if a device vanishes, the broker posts its goodbye message

MQTT = the rules. Mosquitto = a free broker program that follows those rules. It runs on the Raspberry Pi and acts as the mail room.

One Message, Step by Step

1 ESP32 → Broker
Joins the mail room: "I am here."

2 Pi → Broker
Signs up for the readings mailbox: "Send me every reading."

3 ESP32 → Broker
Posts a reading: "Room 2.306 is 27.4 °C."

4 Broker → Pi
Hands the Pi a copy of that reading.

5 Pi → Broker → ESP32
Posts a command: "Cooling ON." The broker passes it on.

6 ESP32 → Broker → Pi
Posts "Done." The Pi now knows it worked.

The sender and receiver never talk directly. The broker sits in the middle.

Naming the Mailboxes (Topics) and Who May Use Them

```
room/2.306/telemetry   readings
room/2.306/command     orders
room/2.306/status      state, "done"

room/+/telemetry     any one room
room/#               everything under room
```

Naming rules

Go from general to specific. Keep readings, commands, and status in separate mailboxes. No spaces or secrets in names.

Who	May send to	May read from
esp32-2.306	room/2.306/telemetry room/2.306/status	room/2.306/command
command service	room/2.306/command	room/2.306/status
dashboard	(nothing)	room/+/telemetry room/+/status

Only what is needed

If one sensor is hacked, it must not be able to send commands to other rooms. The broker enforces these permissions.

+ = one level, # = all levels below (OASIS MQTT). The table is illustrative; each broker has its own permission format.

Quick Check: From Sensor to Broker

Q1 · What should travel with a sensor number?
A: unit, time, trust level, sender ID, message number.

Q2 · Wi-Fi or MQTT: which is the connection?
A: Wi-Fi carries the signal. MQTT is the messaging on top.

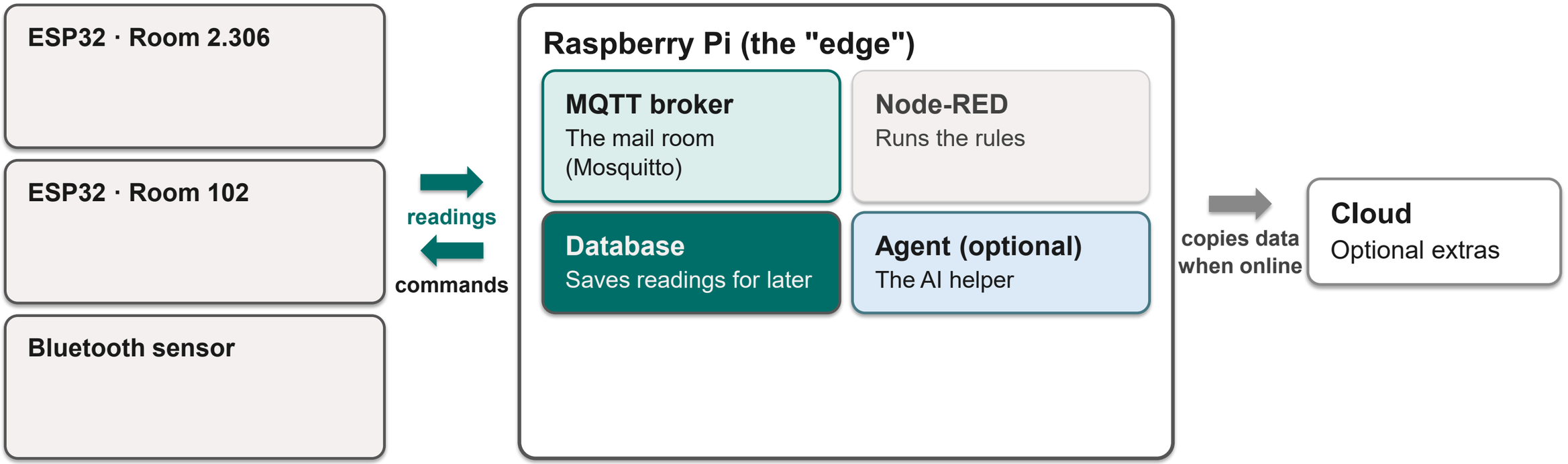
Q3 · A new dashboard starts. What does it see first?
A: the last saved (retained) status message.

Q4 · Wi-Fi drops. What does the ESP32 do?
A: keeps reading, holds a safe setting, keeps trying to reconnect.

Next: the Raspberry Pi makes decisions near the room, so we do not depend on the cloud.

Raspberry Pi: A Small Computer Near the Room

Doing the thinking near the devices, instead of far away in the cloud, is called edge computing.



Fast
Decisions are made in the building. No trip over the internet.

Private
Raw data can stay on site.

Limited
A small computer: watch its memory and storage.

Keeps working
If the internet fails, rules still run. Data is sent later.

If the Internet Fails: Save Now, Send Later

1 Normal
Readings go to the broker, the local database, and the cloud.

2 Outage
Cloud link is down. Rules keep running. The Pi saves readings with their times.

3 Catch-up
The Pi sends saved readings, oldest first. The cloud skips repeats using device ID and message number.

How much space?
One reading every 5 seconds = 17,280 a day. At about 150 bytes each, that is roughly 2.6 MB per sensor per day.

Plan ahead
How long to keep data, and what to delete first when the disk is full. Old commands are never replayed.

The room stays comfortable during an outage; only the dashboard is late.

Arithmetic: 86,400 s ÷ 5 s = 17,280; × 150 B ≈ 2.6 MB. Payload size is an assumption.

Lecture Overview

- 1 The Basics
  The room, sensors, readings, and the ESP32 chip.
- 2 Messaging
  How devices talk (MQTT) and the Raspberry Pi nearby.
- 3 Automation
  Simple rules, where they fall short, and AI.
- 4 Agents
  AI agents, small vs big AI, the full system.
- 5 Practice
  Staying safe, a full worked example, what to do next.

Node-RED: Rules You Draw as a Flow

Node-RED lets you draw rules as boxes joined by wires. Each box does one small job.

1 Receive
Get the latest reading from the broker.

→

2 Clean up
Convert °C to °F.

→

3 Decide
People inside AND above 80 °F?

→

4 Prepare
Write the message "COOL_ON".

→

5 Send
Post it to the command mailbox.

↓ If not: do nothing (nothing is sent)

Why people like it

The same input always gives the same result. You can watch every message. Mistakes are easy to find.

Words to know

Node = one box. Flow = boxes joined by wires. Message = data moving along the wires.

Works with AI

Node-RED can call an AI service, and can check and log what an AI agent suggests.

Where Simple Rules Fall Short

Switching too fast
Temperature hovers near 80 °F and the switch flips on and off. Fix: on at 80 °F, off at 78 °F. The gap is called hysteresis.

Trusting broken data
A stuck sensor says 71 °F all day. Fix: check that readings are recent and believable.

Missing the big picture
Same threshold for an empty hall, an exam, or a hot afternoon. A rule cannot read the timetable.

Too many rules
Dozens of rules for rooms, hours, and seasons start to clash, and nobody knows which one wins.

Fix the rules first. Add AI only for what is left.

From Plain Rules to AI

	Room 2.306 example	Strength	Limit
Rules fixed if-then steps	If people are inside and it is above 80 °F, turn cooling on.	Easy to predict and check	Only knows the numbers you gave it
Machine learning software that learns patterns from past data	Guess the temperature 30 minutes ahead. Spot a sensor that is going wrong.	Finds patterns people miss	Needs data. Can be wrong.
Generative AI software that answers in words	Explain why the room is hot, using the timetable, sensors, and cooling status.	Handles messy situations	Answers may vary. Needs safety checks.

Use rules when

The problem is simple and stable. Start here.

Add AI when

Many things interact, or the situation is unclear.

Lecture Overview

- 1 The Basics
  The room, sensors, readings, and the ESP32 chip.
- 2 Messaging
  How devices talk (MQTT) and the Raspberry Pi nearby.
- 3 Automation
  Simple rules, where they fall short, and AI.
- 4 Agents
  AI agents, small vs big AI, the full system.
- 5 Practice
  Staying safe, a full worked example, what to do next.

What Is an AI Agent?

An agent is software that works toward a goal in a loop. It has a goal, memory of earlier steps, tools, and feedback. Next slide: how this differs from plain rules.

1 Look
Read sensors and history.

2 Think
An AI model (generative AI, slide 22) suggests the next step.

3 Act
Use one approved tool.

4 Check
Did it work? Write it down. Repeat.

Not an agent
A chatbot that answers once. No tools, no loop.

Agents need limits
They can only use tools we approve. The AI never controls hardware directly.

Rules vs AI Agent

	Rules (Node-RED)	AI agent
Who writes the logic?	A person, fixed in advance	The AI picks the next step toward a goal
What it looks at	Set fields in a message	Goal, memory, history, tool results
Same input twice	Same result	Result may differ
How it runs	Once through the flow	In a loop: look, think, act, check
When it fails	In predictable ways; easy to test	May misunderstand; needs checks
Best for	Simple, stable conditions	Unclear situations; explaining and diagnosing

The agent works beside the rules, not instead of them. Rules stay the safety floor.

Agent Tools: A Short, Approved List

Tool	What it can do
get_history	Read past temperatures
get_occupancy	Read: are people inside?
get_hvac_status	Read: is cooling running?
set_cooling	Change: cooling on/off, only after the safety check (slide 31)
anything else	Not allowed (shell, raw MQTT, delete files)

```
{
 "tool": "set_cooling",
 "args": {"room": "2.306",
          "state": "ON",
          "ttl_s": 900},
 "reason": "people inside, 81 F
           for 15 min, cooling off"
}
```

Fixed form
The request has set fields, so code can check it.

Few powers
Many ways to look, one way to act. ttl_s = seconds until the command expires.

Reason saved
The agent says why, and we keep it for review.

Small AI on the Pi, or Big AI in the Cloud?

	Small model on the Pi (SLM)	Big model in the cloud (LLM)
Privacy	Data stays in the building	Data leaves the building; check the rules
Speed	No internet trip, but the Pi is slow	Internet trip, but powerful servers
Internet	Works offline	Needs the internet
Cost	Hardware and power	Pay per use
Ability	Simpler tasks	Harder tasks and wider knowledge

SLM = small language model. LLM = large language model. Whether a model runs on a Pi depends on its size and the Pi's memory. Test it yourself.

1 Rules
Always on. Predictable.

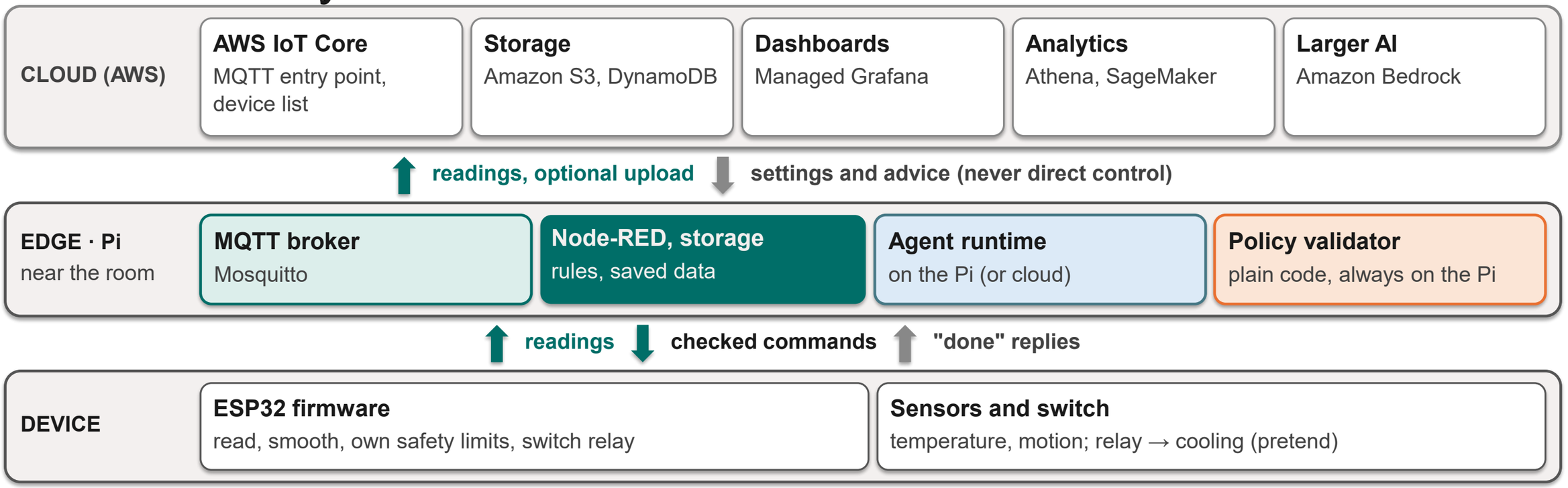
→

2 Small AI on the Pi
Optional, if the Pi can run it.

→

3 Cloud AI
When allowed and online.

The Whole System in One Picture



Quick Check: Rules, AI and Agents

Q1 · Rules or AI: which do you try first?
A: Rules. Add AI only when rules fall short.

Q2 · What four steps does an agent repeat?
A: Look, think, act, check.

Q3 · What may the agent use?
A: Only a short list of approved tools.

Q4 · Small AI on the Pi or big AI in the cloud?
A: Pi: private, works offline, less able. Cloud: more able, needs internet, data leaves the building.

Next: keeping the agent safe, the last big idea.

Lecture Overview

- 1 The Basics
  The room, sensors, readings, and the ESP32 chip.
- 2 Messaging
  How devices talk (MQTT) and the Raspberry Pi nearby.
- 3 Automation
  Simple rules, where they fall short, and AI.
- 4 Agents
  AI agents, small vs big AI, the full system.
- 5 Practice
  Staying safe, a full worked example, what to do next.

Keeping the Agent Safe

1 Suggest
The agent asks to use an approved tool.

2 Check
Plain code asks: sensible value? too often? already sent?

3 Do
The ESP32 checks again. The command expires if not done in time.

4 Confirm
Did the temperature drop? Write it in a log.

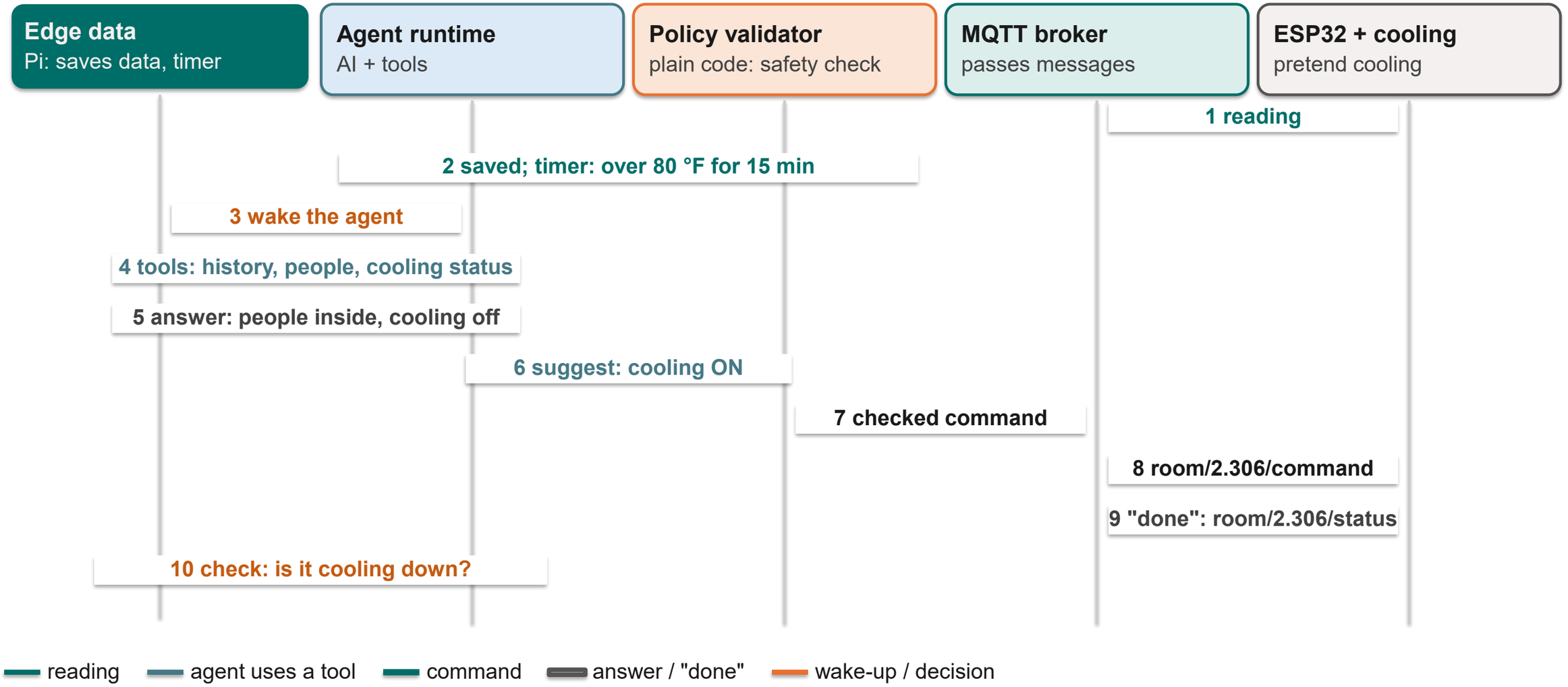
Basic security
Every device has its own identity and password. Messages are encrypted (TLS). Each device may only use its own mailboxes. Check every input and keep logs.

If this fails	Do this
The AI model	Pause the agent. Normal rules keep running.
Wi-Fi	ESP32 keeps a safe setting.
A sensor	Bad or old data: do nothing and raise an alert.

Safety-critical decisions are made by plain code, never by the AI.

Example: A Hot Room With People In It

Scenario: people are inside, it has been above 80 °F (26.7 °C) for 15 minutes, and cooling looks switched off.



How Much Freedom Does the Agent Get?

L0 · Watch only
Dashboards and alerts.

L1 · Suggest
The agent proposes; a person approves.

L2 · Act within limits
Small, reversible actions such as cooling, after the safety check.

L3 · Act freely
Not for devices that affect the physical world.

Start at L1. Move cooling to L2 once tests pass.

Replay old data through the agent with a pretend switch.

Try to trick it: hostile text in a device name must not become a command (prompt injection).

Count mistakes, response time, and how often people step in.

Kill switch: one control pauses the agent. Rules keep running.

What You Can Now Do

Follow data from sensor to cloud
Sensor → ESP32 → MQTT → Pi → cloud (slides 5–10, 17, 28).

Explain MQTT
Broker, mailboxes (topics), delivery levels, saved and goodbye messages, permissions (slides 13–15).

Tell rules from agents
Fixed rules vs an AI that loops toward a goal (slides 20–26).

Explain safe control
Suggest, check, do, confirm, with backups for failure (slides 31–33).

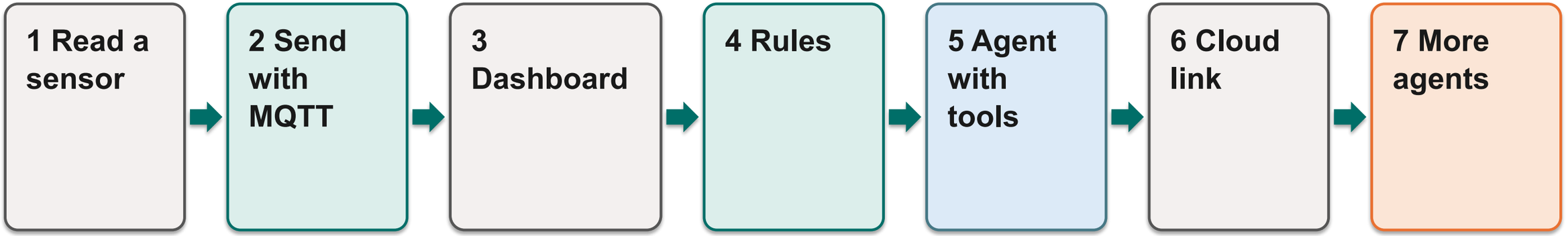
Lab 1
A pretend ESP32 sends readings to Mosquitto.

Lab 2
A Node-RED rule with a gap (hysteresis) sends a command.

Lab 3
Give an agent one read-only tool and save its reasons.

What Comes Next

If you build the Room 2.306 system yourself, go in this order:



If you want AI agents, three agent jobs can be split up:

Watcher

Reads data and flags problems. Cannot act.

Investigator

Explains likely causes from history. Cannot act.

Controller

May ask for commands, only through the safety check.

One may be enough

More agents mean more messages and more ways to fail.

Discuss: which Room 2.306 decisions should never be left to an AI? Why?

Thank you

From one sensor reading to a safe, smart AI agent.

Start simple
Sensors, messages, and plain rules come first.

Add AI with care
Use it only where rules fall short.

Keep control safe
The AI suggests. Plain code checks. The ESP32 acts.